
CODE REVIEW REPORT

Project: *cypressAllure*

Repository: <https://github.com/AZANIR/cypressAllure>

Date: 24 June 2026

Reviewer: Ernest Sosnovskyi

Executive Summary This report presents a comprehensive code review of the *cypressAllure* automated testing framework. The analysis evaluates CI/CD pipeline configurations, Page Object Model architecture, test data management, and version control practices. While the repository establishes a basic end-to-end testing structure, several critical security vulnerabilities, legacy anti-patterns (pre-Cypress 10), and fragile test implementations were identified. The detailed breakdown below outlines these findings and provides industry-standard recommendations to enhance the scalability, security, and stability of the framework.

1. Uninformative Commit Messages: The repository's commit history contains vague and non-descriptive messages (e.g., *"final update yml"*, *"up"*, *"update test"*, *"update readme"*, *"Deploying to master from master c5de30f"*). This practice severely hinders repository traceability, making it difficult for team members to understand the context or intent of historical changes. Commit messages should adhere to a standardized convention (such as Conventional Commits), clearly detailing *what* was changed and *why* (e.g., *test(login): refactor valid credentials test to use fixture data*).

2. GitHub Actions: [.github/workflows/cyAllureRun.yml](#)

- **End-of-Life Node.js Version:** The matrix strategy specifies *node-version*: *[14.17.0]*. Node.js 14 is completely deprecated and no longer receives security updates. This must be updated to a supported LTS version (e.g., 18.x or 20.x).
- **Deprecated Actions Versions:** The workflow relies on outdated GitHub Actions:

- actions/checkout@v2 (should be v4)
- actions/setup-node@v2 (should be v4)
- actions/upload-artifact@v1 (should be v4) Using v1/v2 actions can lead to workflow failures due to Node 12 deprecations in GitHub Runners.
- **Syntax Error in Secrets:** The deployment step contains a critical syntax error: ACCESS_TOKEN: \${{'secrets.ACCESS_TOKEN'}}. The quotation marks inside the curly braces will evaluate to a literal string rather than reading the actual repository secret. The correct syntax is \${{ secrets.ACCESS_TOKEN }}.
- **Redundant Dependencies Installation:** The step run: npm i npm@latest is unnecessary, increases the pipeline execution time, and can introduce unexpected package manager behaviors. Standard npm ci should be used instead of npm i for clean, reliable CI builds.
- **Outdated Third-Party Action:** The deployment action JamesIves/github-pages-deploy-action@3.1.0 is heavily outdated (current versions are 4.x+).

3. Configuration: config/config.config.ts

- **Incorrect Naming Convention:** The configuration file is named config.config.ts. By default, Cypress expects the configuration file to be named cypress.config.ts. Deviating from this standard requires passing explicit CLI flags during execution.
- **Legacy Migration Artifacts:** The setupNodeEvents block utilizes a require() statement to import cypress/plugins/index.js alongside an auto-generated comment (// We've imported your old cypress plugins here). This is an outdated migration stub from Cypress 9 to 10+. It breaks TypeScript's strict typing by importing a CommonJS module and preserves deprecated architecture. Plugin logic should be implemented natively within this configuration file.
- **Redundant Timeout Configurations:** Explicitly defining properties such as defaultCommandTimeout: 5000, execTimeout: 5000, and requestTimeout: 5000 is unnecessary as these match the default Cypress values. This creates configuration clutter.
- **Insecure Base URL Protocol:** The baseUrl uses an unencrypted HTTP protocol (http://automationpractice.com/index.php). It is recommended to test against HTTPS endpoints to mirror modern production environments.

4. Page Object Model: [cypress/e2e/pages/loginPage.ts](#)

- **Unmasked Sensitive Data:** In the [login](#) method, the password is typed directly using [this.passwordTxt.type\(password\)](#). This will log the password in plain text within the Cypress Command Log, test runner interface, and video recordings. To ensure security, it must be masked using the [{ log: false }](#) option: [this.passwordTxt.type\(password, { log: false }\)](#).
- **Missing Actionability & Pre-checks:** The [login\(\)](#) method interacts with inputs ([.type\(\)](#)) without asserting their readiness or clearing existing values. This is a common source of flaky tests. Best practice dictates checking visibility and clearing inputs before typing: [this.emailAddressTxt.should\('be.visible'\).clear\(\).type\(emailId\)](#).
- **Fragile Selectors:** The locators rely heavily on generic CSS classes and IDs (e.g., [.login](#), [#email](#)). These are tied to styling and application logic, making them prone to unexpected changes. The industry standard is to use dedicated testing attributes such as [\[data-cy="login-button"\]](#) or [data-testid](#).
- **Cypress Anti-pattern (Getters):** The use of ES6 getters ([get signinLink\(\)](#)) in Cypress Page Objects is widely debated. Cypress commands are asynchronous and queued, whereas getters execute synchronously upon access. While functional in basic scenarios, it is safer to use standard methods (e.g., [signinLink\(\) { return cy.get\('.login'\) }](#)) to prevent unexpected queue behaviors in complex tests.

5. Page Object Model: [cypress/e2e/pages/myAccountPage.ts](#)

- **Tight Coupling (Cross-Importing):** The [MyAccountPage](#)-class imports the [loginPage](#) instance specifically to validate the logout action ([loginPage.signinLink.should\('be.visible'\)](#)). This is a significant architectural anti-pattern. Page Objects should remain isolated and only contain logic relevant to their specific page context. Validating page transitions or asserting elements on *another* page should be orchestrated within the test spec file itself.
- **Missing Actionability Assertion:** The [logout\(\)](#) method executes a [.click\(\)](#) command directly on the sign-out link. It is safer to ensure the link is visible and interactable first: [this.signoutLink.should\('be.visible'\).click\(\)](#).
- **Fragile Text Assertion:** The method [validateSuccessfulLogin\(\)](#) uses [.should\('have.text', 'My account'\)](#). Exact text matching can be brittle due to unexpected whitespaces or case changes. Using [.should\('include.text', 'My account'\)](#) or [.should\('contain.text', ...\)](#) is generally more robust.

- **Locator Strategy & Getters:** As noted in the Login Page review, the use of generic CSS classes (.logout, .page-heading) and ES6 getters should be reconsidered in favor of data-* attributes and standard methods.

6. Test Implementation: cypress/e2e/tests/login.test.ts

- **Mocha Context Binding (Arrow Functions):** The describe and beforeEach blocks use arrow functions (() => {}). Cypress and Mocha strongly advise against using arrow functions because they bind the lexical this context. When using cy.fixture().then(function (data) { this.data = data; }), the this context might not reliably share data with the it blocks. All describe, beforeEach, and it blocks should be refactored to use standard function() declarations.
- **Test Case Duplication & Hardcoding:** The first test (login with valid credentials) utilizes hardcoded email and password strings. The subsequent test (... read data from fixture) executes the exact same flow using fixture data. The hardcoded test is redundant, creates maintenance overhead, and should be completely removed.
- **Assertion Typo:** The final test expects the error message 'Invalid email addressssss.'. This obvious typographical error will result in a false test failure or, worse, validate an existing bug in the application.
- **Missed Opportunity for Data-Driven Testing:** Writing separate it blocks for every invalid login scenario (invalid email, invalid password, wrong format) leads to code bloat. Since the data is already stored in a fixture file, these tests should be dynamically generated using a simple forEach loop over the invalid credentials object.

7. Test Implementation: cypress/e2e/tests/myAccount.test.ts

- **External Domain Navigation:** The beforeEach hook visits an external website (https://google.com). E2E tests should strictly test the target application using the configured baseUrl. Navigating to third-party domains outside the scope of the project is an anti-pattern.
- **Dead Code & Incomplete Implementation:** The legitimate application setup command (loginPage.launchApplication()) is commented out. Combined with a test named 'Sample Test' that contains zero assertions, this entire file appears to be abandoned draft code that provides no test coverage and clutters the repository. It should be removed or properly implemented.
- **Synchronous Logging in Asynchronous Tests:** The test utilizes native console.log(). Cypress commands run asynchronously and are queued. A native console.log evaluates synchronously in the Node/Browser context and

will print to the browser console immediately, bypassing the Cypress command log interface. For proper test execution tracking, `cy.log()` must be used.

- **Mocha Context Binding (Arrow Functions):** Similar to the login tests, this file incorrectly uses arrow functions for `describe`, `beforeEach`, and `it` blocks, which prevents proper `this` context binding in Mocha.

8. Test Data Management: `cypress/fixtures/users.json`

- **Security Vulnerability (Plain Text Secrets):** Storing sensitive data, such as passwords ("`Test@1234`"), in plain text within the repository is a critical security anti-pattern. Even for test environments, credentials should never be committed to version control. Passwords should be managed securely using environment variables (`Cypress.env`), a `.env` file (added to `.gitignore`), or CI/CD secrets managers (like GitHub Secrets).
- **Test Variable Isolation:** In the `wrong_email_format` object, the password provided is "`test@12345`" (which is the invalid password from the previous test case). When testing a specific negative scenario (e.g., wrong email format), it is a best practice to keep all other variables valid (use the valid password). This isolates the variable being tested and ensures the system rejects the login specifically due to the email format, not the password.

9. Legacy Architecture: `cypress/plugins/index.js`

- **Deprecated File Structure:** The `plugins/index.js` file is obsolete in Cypress version 10 and above. Maintaining a separate `plugins` directory is considered a legacy anti-pattern. The plugin initialization logic (in this case, the `@shelex/cypress-allure-plugin/writer`) must be migrated directly into the `setupNodeEvents(on, config)` function within the main `cypress.config.ts` file.
- **Type Safety Breakage:** The project utilizes TypeScript, yet this legacy plugin file uses CommonJS (`module.exports` and `require`). This breaks the TypeScript compilation chain and type inference. By moving this configuration directly into the typed `cypress.config.ts`, the framework will maintain strict type safety across all configuration levels.
- **Redundant Abstraction:** Keeping this file only to export a single Allure writer function creates unnecessary abstraction. Deleting the `plugins` folder entirely after migration will clean up the project structure.

10. Cypress Support Files: `cypress/support/commands.js` & `./support/e2e.js`

- **Inconsistent File Extensions:** The project relies on TypeScript for its Page Objects and test specifications, yet the support files are maintained as

JavaScript (.js). To ensure a robust architectural standard, commands.js and e2e.js should be renamed to commands.ts and e2e.ts.

- **Repository Noise (Boilerplate Code):** The commands.js file consists entirely of auto-generated instructional comments from the Cypress initial setup. Committing unedited, placeholder boilerplate code creates unnecessary visual noise in the repository. If no custom commands are currently required, the file should either be cleared of comments or safely deleted (removing its import from e2e.js).

11. Repository Structure: docs Directory

- **Generated Artifacts in Source Control:** The docs directory contains the auto-generated Allure report assets (HTML, CSS, JS), as dictated by the deployment step in the GitHub Actions workflow. Committing generated build artifacts or test reports directly to the main source code branch (master) is a significant version control anti-pattern. It drastically inflates the repository size, creates massive amounts of unreadable code diffs, and frequently causes merge conflicts.
- **Recommended CI/CD Architecture:** The GitHub Actions deployment step should be refactored to push the Allure report to an isolated, dedicated branch (typically named gh-pages), completely separating the source code from the deployment artifacts.
- **Ignore Configuration:** The docs/ (or allure-report/) directory must be strictly added to the .gitignore file to ensure developers do not accidentally commit locally generated test reports to the remote repository.

12. Version Control Configuration: .gitignore

- **Missing Environment Secret Files (Security Risk):** The file lacks critical entries for local environment configuration files, such as .env or cypress.env.json. To mitigate the security vulnerabilities identified in the users.json fixture (plain text passwords), sensitive data must be moved to environment files, and those files must be explicitly ignored here to prevent accidental commits.
- **Missing docs/ Directory:** As established in the repository structure review, the docs directory acts as a dumping ground for generated Allure artifacts. It must be added to .gitignore to prevent developers from accidentally pushing their local test reports into the version control history.
- **Redundant Syntax Rules:** The file includes both reports and reports/. In Git, ignoring a directory (e.g., reports) automatically ignores all of its contents. The / wildcard rules are redundant and create unnecessary visual clutter. The rules for allure-report/ and allure-results/ should similarly be simplified to just the directory names.

13. Project Configuration & Dependencies: package.json

- **Cross-Platform Incompatibility:** The clear script utilizes the native Unix command rm -r reports/ || true. While this executes successfully in Linux/WSL or macOS environments, it will fail natively on standard Windows machines. For cross-platform compatibility in team environments, it is standard practice to use an npm package like rimraf (e.g., rimraf reports/).
- **Inconsistent Path Configurations:** There is significant fragmentation in directory paths for Allure reports. The report script points to reports/ui/allure-results, the allure:generate script points to allure-results, and the config.config.ts points to ../allure-results. This misalignment will inevitably cause pipeline or local report generation to fail.
- **Environment & Typing Mismatch:** The @types/node dependency is set to ^18.0.5 (Node 18), which severely contradicts the .github/workflows pipeline that explicitly requests Node 14.17.0. Local and CI/CD environments must be strictly aligned.
- **Outdated Major Dependencies:** The project relies on cypress version 10.3.0, which is significantly outdated and misses numerous stability updates and modern API features. Furthermore, the @shelex/cypress-allure-plugin is a legacy package; modern frameworks generally migrate to newer Allure integrations.
- **Formatting Artifact:** The description field contains escaped quotes ("\"Automation Testing framework...\""). This is visually untidy and should be a standard string without escaped internal quotes.

14. Dependency Lockfile: package-lock.json

- **Deterministic Builds:** The package-lock.json (v2) is correctly committed to the repository. This is a critical best practice to ensure deterministic, reproducible builds across different local machines and the CI/CD pipeline.
- **Pipeline Optimization Opportunity:** Because the lockfile is properly maintained, the GitHub Actions workflow must be updated to use npm ci (Clean Install) instead of the currently configured npm i. The npm ci command strictly bypasses package.json to install exact versions from the lockfile, which is faster and prevents unintended minor version upgrades during automated test runs.
- **Security & Auditing:** Since manual review of auto-generated lockfiles is out of scope, it is highly recommended to enable automated security scanning (such as GitHub Dependabot or executing npm audit in the pipeline) to monitor the dependency tree for known vulnerabilities.

15. Documentation: README.md

- **Formatting Errors:** The primary header contains concatenated markdown formatting (`# cypress-e2e-allure# Cypress-e2e-Allure...`), making it visually untidy.
- **Fragile Asset Hosting:** The repository documentation relies on external image hosting services (imgur.com) for its screenshots. If these external links expire or are deleted, the documentation will break. Images should be stored natively within the repository (e.g., in an `/assets` or `/docs/images` folder) and referenced via relative paths.
- **Unflattering Showcase:** The screenshot provided to demonstrate the "Allure report" displays 1 passing test and 5 broken/skipped (yellow) tests. Using a predominantly failing/broken test suite as the primary showcase in documentation reflects poorly on the framework's reliability.
- **Incomplete Setup Instructions:** * The `git clone` instruction is missing the actual repository URL.
 - The prerequisite Node.js version (e.g., v18 or v20) is not documented.
 - The instruction to "Change Username and Password" is overly vague. It does not specify *where* these credentials should be updated (e.g., in a `.env` file or `users.json`).
 - While the execution script (`npm run cy:run`) is provided, the instructions completely omit how to generate or view the Allure report locally (e.g., `npm run report`), which defeats the purpose of an "Allure framework" repository.

16. TypeScript Configuration: `tsconfig.json`

- **Broken Path Resolution:** The `include` array contains paths navigating outside the current directory (`"../node_modules/..."`). Assuming `tsconfig.json` is located in the project root, navigating up a directory will fail to locate the `node_modules` folder, breaking type definitions for Cypress and Allure.
- **Improper Type Inclusion (Anti-pattern):** Hardcoding specific paths to `node_modules` within the `include` array is a TypeScript anti-pattern. Third-party type definitions should be managed via the `types` array under `compilerOptions`. The Allure plugin types should be added as `"types": ["node", "cypress", "@shelex/cypress-allure-plugin"]`, and the `include` array should be simplified to `"include": ["/*.ts"]`.
- **Outdated ECMAScript Target:** The compilation target is set to `"es2016"`. Considering the project attempts to use Node 18 (as seen in `package.json`), targeting `"es2022"` or `"esnext"` is highly recommended to leverage modern JavaScript features and improve compilation efficiency.

Conclusion

The evaluated *cypressAllure* repository demonstrates a foundational understanding of E2E automation and Allure integration, but it currently relies heavily on outdated architecture and deprecated CI/CD workflows. Critical improvements are necessary regarding credential security, Page Object Model encapsulation, and overall test resilience. By addressing these architectural flaws, eliminating legacy code, and adhering to modern TypeScript and Cypress best practices, the framework can be significantly optimized for reliable continuous integration and scalable team collaboration.